

Power BI in Fabric

**Common Patterns.
Clear Fixes.**

50 engineering patterns covering Direct Lake mode, semantic model design, DAX performance, row level security, report performance, deployment pipelines, and Power BI behavior specific to Microsoft Fabric. Patterns every Fabric engineer must recognise, covering the gap between how Fabric is described and how it actually behaves.

INTRODUCTION

Power BI changed. Most people did not notice.

Power BI inside Microsoft Fabric is not the same product as standalone Power BI. Direct Lake storage mode, the two flavors of it that now ship side by side, framing instead of refresh, RLS that routes differently depending on where it is defined, capacity guardrails that are silent when crossed. None of these behaviors exist in the classic Power BI service.

The most common source of engineering problems in Fabric Power BI is engineers applying classic Power BI knowledge to a system that has moved on. Import mode assumptions applied to Direct Lake models. SQL endpoint RLS patterns applied to a model that needed semantic-model RLS. Desktop-tested DAX measures that behave differently in the service because the storage mode silently fell back. These are not bugs. They are the gap between what engineers know and what Fabric actually does.

This book is a reference, not a tutorial. Each of the fifty scenarios is self-contained. You can read them in order, or jump to the symptom you are actually seeing using the diagnostic index on the next page. You do not need to have read Books I to III to use Book IV. The seven chapters cover Direct Lake mode, semantic model design, DAX performance, Row Level Security, report performance, deployment pipelines, and the specific behaviors that surface when Power BI connects to Fabric data sources.

Every scenario follows the same structure: the situation, the wrong thinking, what is actually happening, and the fix. The wrong thinking section matters as much as the fix. Understanding why the wrong answer is tempting is the difference between solving a specific problem and building the intuition to avoid the next one.

Reflects the state of Microsoft Fabric as of Q2 2026, after the GA of Direct Lake on OneLake and the F-SKU migration from Premium. Behaviors marked as current may change.

Sanjay Chandra

Introduction

Power BI in Fabric • 2

DIAGNOSTIC INDEX

Start with the symptom. Not the chapter.

When something is broken in production, the symptom is what you have. Find it below, check the scenarios in the order listed, and stop at the one that matches.

Report was fast, now it is slow.

Check **01** (schema change), **06** (framing stopped), **28** (RLS triggered fallback), **31** (Desktop vs service).

Data in the lake but not in the report.

Check **02** (framing timing), **46** (refresh ordering), **49** (shortcut visibility), **50** (blank after refresh).

Works in Desktop, breaks in service.

Check **26** (UPN resolution), **18** (time intelligence context), **38** (deployment rules), **43** (external users).

RLS made everything slow.

Check **28** first (most common), then **27** (filter complexity), **30** (OneLake security path).

DAX measure is wrong, not slow.

Check **17** (CALCULATE filter logic), **21** (visual context), **22** (ALLSELECTED), **11** (date table), **48** (composite totals).

Page hangs on load or on slicer change.

Check **32** (matrix cardinality), **33** (visual fan-out), **34** (single card slow), **20**.

Storage mode says Direct Lake but it feels like DirectQuery.

Check **03** (fallback diagnosis), **01**, **04** (SQL view), **05** (programmatic add), **28** (RLS path).

Refresh succeeded, results still wrong.

Check **02**, **42** (deployment cache), **50**, **11** (date table not marked).

RLS lets users see data they should not.

Check **24** (role logic), **25** (disconnected fact), **29** (multi-role union), **26** (UPN format).

DAX measure is slow.

Check **19** (SUMX vs pre-agg), **16** (iterator size), **20** (FILTER scope), **23** (RANKX ties).

Different users see different results.

Check **35** (per-user cache), **43** (external view access), **26**, **29**.

Dev to prod promotion did damage.

Check **38** (connections), **39** (pointing at dev), **41** (downstream breakage), **42** (stale data).

Several scenarios appear under multiple symptoms because one cause produces several symptoms. Check the highest-listed scenario first. If your symptom is not above, the table of contents is organized by cause.

FABRIC ENGINEERING PATTERNS

Chapter 1

Direct Lake Mode

Eight scenarios on how Direct Lake on SQL and Direct Lake on OneLake behave differently in practice, why fallback to DirectQuery happens silently on one but not the other, and which diagnostic surfaces actually tell you the truth.

01

DIRECT LAKE MODE

Your Direct Lake on SQL report was fast. After a Lakehouse schema change it fell back to DirectQuery silently.

THE SITUATION

A Direct Lake on SQL semantic model has been performing well for weeks. A data engineering team adds new columns to the underlying Lakehouse Delta table as part of a routine schema evolution. After the change, report visuals load noticeably slower. No error is shown anywhere and the model still shows Direct Lake as the storage mode in the service.

WRONG THINKING

Engineers assume the slowdown is capacity-related or a temporary platform issue. Since no error is visible and the model label still shows Direct Lake mode, the schema change is not connected to the performance regression. DAX Studio is not opened to verify the actual execution path.

WHAT IS ACTUALLY HAPPENING

The schema change caused a framing failure during the next automatic refresh. When Direct Lake on SQL cannot successfully frame all columns, it falls back to DirectQuery mode silently. The model label in the service still shows Direct Lake but every query is now going through the SQL analytics endpoint. DirectQuery performance is significantly slower because each visual triggers a SQL query to the endpoint rather than reading from in-memory VertiPaq column segments. The fallback is invisible unless you check query traces in DAX Studio or the Performance Analyzer.

THE FIX

After any schema change to the underlying Lakehouse or Warehouse, manually trigger a semantic model refresh and verify Direct Lake mode is active. The fastest signal is the model property `DirectLakeBehavior`. During testing, set it to `DirectLakeOnly` so that fallback becomes a visible error instead of a silent slowdown; leave it on `Automatic` only in production once the model is known to be stable. To inspect the current state, use DAX Studio server timings or the `INFO.STORAGES()` and `INFO.VIEW.TABLES()` DMVs to check storage mode per table. Common fallback causes are unsupported column data types, tables that exceed the F-SKU guardrails for rows or row groups, and tables that never reframed after the schema changed. Never assume the storage mode label in the service reflects the actual query execution path.

02

DIRECT LAKE MODE

Your Direct Lake semantic model is not reflecting the latest Lakehouse data despite a successful refresh.

THE SITUATION

A pipeline loads new data into a Lakehouse Delta table. The semantic model refresh completes successfully and the service shows a recent refresh timestamp. But Power BI reports still show the previous day's data. Refreshing the browser and clearing the cache makes no difference. Querying the Lakehouse table directly confirms the new data is there.

WRONG THINKING

Engineers assume the semantic model refresh did not fully complete or that a caching layer in the Power BI service is serving stale data. They trigger multiple manual refreshes hoping one will propagate the new data. The framing concept specific to Direct Lake is not considered because it looks and feels like a normal refresh operation.

WHAT IS ACTUALLY HAPPENING

Direct Lake uses a process called framing to determine which version of the Delta table the semantic model reads. A Direct Lake refresh does not copy data. It copies only metadata about the current Delta table state. If the framing operation ran before the pipeline finished writing all Delta transaction log entries, the model is framed to the previous table version. The refresh status shows success because framing itself succeeded. The model is simply framed to an older snapshot and new data written after the framing point is invisible until the next refresh.

THE FIX

Ensure the semantic model refresh is triggered only after the Lakehouse pipeline has fully committed all data. Add the semantic model refresh as the final step in the pipeline rather than scheduling it independently on a fixed time that may overlap with a variable-length data pipeline. Use the Power BI REST API or a Fabric notebook activity to trigger the refresh after the data load confirms completion. The model refresh must always happen after the data write is complete, not concurrently with it.

03

DIRECT LAKE MODE

Your Direct Lake on SQL model fell back to DirectQuery. You do not know why and there is no clear error.

THE SITUATION

A Direct Lake on SQL model that was performing well starts running slowly. No error appears in the service. The model refresh succeeds. The storage mode label still shows Direct Lake. Users report that report pages that used to load in 2 seconds now take 15 to 20 seconds. The engineering team has no idea where to start diagnosing.

WRONG THINKING

Engineers check the Lakehouse for data issues, review recent pipeline runs, and check Fabric capacity metrics. When nothing obvious appears, the slowdown is attributed to a platform change or infrastructure issue. The execution mode of the model itself is not verified because it appears correct in the UI.

WHAT IS ACTUALLY HAPPENING

Direct Lake on SQL has several conditions that trigger silent fallback to DirectQuery: the number of Parquet files or row groups exceeding the SKU-based limits, model tables based on SQL views rather than Delta tables, RLS or OLS defined at the SQL analytics endpoint, and unprocessed tables after a schema change. Any of these conditions causes the entire model or specific tables to fall back without surfacing a visible error to users or administrators. The fallback is by design to keep reports working, but the performance penalty can be severe.

THE FIX

Connect to the semantic model using DAX Studio and run `EVALUATE INFO.VIEW.TABLES()` to see the storage mode of each table. Tables showing DirectQuery instead of Direct Lake have fallen back. Check the fallback reason using the `SSAS_DIAGNOSTICS` DMV or the model properties in the Fabric web experience. Once the reason is identified, fix the root cause: optimize Delta tables to reduce file counts, replace SQL views with Delta tables, or move RLS to the semantic model layer. After fixing, refresh the model to restore Direct Lake execution.

04

DIRECT LAKE MODE

Your Direct Lake on SQL model works correctly on the Lakehouse. It breaks when pointed at a SQL view.

THE SITUATION

A semantic model was built on Delta tables in the Lakehouse and worked correctly in Direct Lake mode. The data engineering team refactored some transformations into SQL views in the SQL analytics endpoint to simplify their pipeline logic. After the semantic model was updated to reference the views, performance dropped significantly and some measures returned unexpected results.

WRONG THINKING

Engineers assume the SQL views produce the same data as the Delta tables they replaced and that the semantic model should behave identically. The performance drop is attributed to the SQL views being more complex than the original Delta tables rather than to a fundamental change in how the semantic model accesses the data.

WHAT IS ACTUALLY HAPPENING

When a Direct Lake on SQL semantic model references a SQL view rather than a Delta table, the engine cannot use Direct Lake mode for that table. SQL views are evaluated at query time through the SQL analytics endpoint using DirectQuery mode, not through the Delta files directly. The semantic model may appear to be in Direct Lake mode for other tables but the view-based table is always in DirectQuery. Any measure that touches the view-based table forces that portion of the query through the SQL endpoint, adding latency and potentially changing results due to timing differences between the Direct Lake and DirectQuery execution paths.

THE FIX

Replace SQL view references in the semantic model with references to the underlying Delta tables. If the transformation logic in the view is needed, materialize it into a Delta table using a notebook or pipeline and reference the materialized table instead. If views cannot be avoided, use Direct Lake on OneLake mode rather than Direct Lake on SQL, since Direct Lake on OneLake does not support views at all and forces a cleaner architecture. Never mix Direct Lake and DirectQuery table references in the same semantic model without understanding the performance implications.

05

DIRECT LAKE MODE

You added a table to your Direct Lake model programmatically. It is always running in DirectQuery mode.

THE SITUATION

A table was added to a Direct Lake semantic model using a third-party tool, TOM scripting, or TMSL. The table definition looks correct and the model deploys without errors. But every query that touches this table is noticeably slower than queries on other tables in the same model. Checking the execution mode reveals the new table is in DirectQuery while all other tables are in Direct Lake.

WRONG THINKING

Engineers review the table definition and confirm it points to the correct Delta table. The table name, schema, and connection string all look identical to other tables in the model that are working correctly in Direct Lake mode. The programmatic addition process is not suspected as the cause.

WHAT IS ACTUALLY HAPPENING

When a table is added to a Direct Lake semantic model programmatically, it must be explicitly processed after addition before Direct Lake can use it. An unprocessed table cannot be loaded into the VertiPaq memory cache because the engine has no column segment metadata for it. Without column segments, Direct Lake falls back to DirectQuery for every query touching that table. The table appears valid in the model definition but the engine treats it as unready for Direct Lake execution until a process operation completes.

THE FIX

After adding any table to a Direct Lake model programmatically, run a full process operation on that specific table before publishing or testing. In TOM scripting, call `model.Tables["tablename"].RequestRefresh(RefreshType.Full)` and save changes. In the Fabric service, trigger a semantic model refresh which will process any unprocessed tables. Verify the table is in Direct Lake mode using `INFO.VIEW.TABLES()` after the refresh completes. Add table processing as a mandatory step in any automated model deployment pipeline.

06

DIRECT LAKE MODE

Direct Lake framing is not happening. Every report query is scanning full Delta tables.

THE SITUATION

A Direct Lake semantic model appears to be working but report query times are consistently slow regardless of visual complexity or filter scope. Even a card visual showing a single row count takes 10 to 15 seconds. The model is confirmed to be in Direct Lake mode and the Delta tables are reasonably sized. No fallback to DirectQuery is detected.

WRONG THINKING

Engineers optimize DAX measures, reduce visual complexity, and check Power BI capacity metrics. Some improvement is seen but the baseline performance remains far below what Direct Lake should deliver. The framing process itself is not investigated because the model refreshes successfully and shows a current timestamp.

WHAT IS ACTUALLY HAPPENING

Direct Lake framing is the process of reading Delta table metadata to determine which column segments to load into the VertiPaq memory cache. If the Delta tables have very large numbers of small Parquet files, the framing process must process metadata for each file individually. With thousands of small files, framing becomes slow and the resulting column segments cover many tiny chunks of data rather than large efficient segments. Queries still execute in Direct Lake mode but the performance resembles a poorly optimized Import model because the underlying file structure was never optimized for columnstore access.

THE FIX

Run **OPTIMIZE** on all Delta tables referenced by the semantic model to compact small files into larger Parquet files. This reduces the number of file metadata entries the framing process must read and produces larger, more efficient column segments in the VertiPaq cache. After OPTIMIZE, trigger a semantic model refresh to reframe against the optimized table structure. Schedule OPTIMIZE as part of the post-load pipeline maintenance so framing efficiency is maintained as data accumulates over time.

07

DIRECT LAKE MODE

You migrated from Direct Lake on SQL to Direct Lake on OneLake. Reports are now failing instead of falling back.

THE SITUATION

A semantic model was migrated from the original Direct Lake on SQL to the newer Direct Lake on OneLake introduced in April 2025. In the original model, when certain capacity limits were exceeded or unsupported features were encountered, visuals would slow down but still show data by falling back to DirectQuery. In the new model, those same conditions now cause visuals to return errors or blank results with no data shown at all.

WRONG THINKING

Engineers assume the migration introduced a configuration error and compare settings between the old and new model. Everything looks similar. The fundamental behavioral difference between the two Direct Lake flavors is not known because both are labeled Direct Lake in the service.

WHAT IS ACTUALLY HAPPENING

Direct Lake on SQL and Direct Lake on OneLake have fundamentally different fallback behavior. Direct Lake on SQL falls back to DirectQuery when limits are exceeded or unsupported features are encountered, so reports continue to show data at the cost of slower performance. Direct Lake on OneLake has no DirectQuery fallback. When it cannot execute in pure Direct Lake mode it fails completely. Any condition that previously caused a graceful slowdown in the SQL model now causes a hard failure in the OneLake model. The two flavors look similar but behave very differently under stress conditions.

THE FIX

Before migrating to Direct Lake on OneLake, audit the existing Direct Lake on SQL model for all conditions that were triggering fallback to DirectQuery. Use DAX Studio to identify which queries were falling back and why. Fix each root cause before migrating: remove SQL views, optimize Delta tables to stay within SKU parquet file limits, move RLS to the semantic model layer rather than the SQL endpoint. Only migrate when the Direct Lake on SQL model is confirmed to be running in pure Direct Lake mode with no fallbacks. A model that relied on graceful fallback will fail hard after migration to OneLake mode.

08

DIRECT LAKE MODE

You have two Direct Lake models on the same Lakehouse. One is consistently fast, the other is always slow.

THE SITUATION

Two semantic models connect to the same Lakehouse and reference overlapping sets of Delta tables. Both models use Direct Lake mode. One model consistently returns report results in under 2 seconds. The other consistently takes 15 to 20 seconds for similar visuals on the same underlying data. The data engineering team has not changed the Lakehouse tables.

WRONG THINKING

Engineers compare the two models and find similar table counts, similar measure complexity, and similar report designs. The performance gap is attributed to different Power BI capacity allocations or a workspace configuration difference. The model designs themselves are not examined closely.

WHAT IS ACTUALLY HAPPENING

The fast model was designed with a star schema: a fact table surrounded by narrow dimension tables with clean single-column join keys. The slow model uses a snowflake schema with multiple levels of dimension tables joined through intermediate tables, and some measures reference tables through chains of 3 or 4 relationships. In Direct Lake mode, the VertiPaq engine must traverse these relationship chains for every query. Each additional relationship hop adds join overhead. A snowflake schema that works acceptably in Import mode, where data is pre-loaded and relationships are pre-computed, shows its full cost in Direct Lake mode where relationship traversal happens at query time.

THE FIX

Flatten the slow model's schema toward a star schema by denormalizing dimension hierarchies into single wide dimension tables. Replace snowflake chains with pre-flattened dimension tables that join directly to the fact table on a single surrogate key. If denormalization is not possible in the semantic model, materialize flattened dimension tables in the Lakehouse and reference those instead of the raw normalized tables. A well-designed star schema in Direct Lake mode will always outperform a snowflake schema because it minimizes relationship traversal at query time.

FABRIC ENGINEERING PATTERNS

Chapter 2

Semantic Model Design

Seven scenarios covering how semantic model design decisions made before writing a single DAX measure determine whether reports are fast or slow in production.

09

SEMANTIC MODEL DESIGN

Your semantic model has a many-to-many relationship. Filter context is behaving unexpectedly.

THE SITUATION

A semantic model contains a many-to-many relationship between a fact table and a dimension table, implemented using a bridge table. Some measures return correct results in isolation but produce wrong totals when combined with slicers from the dimension table. The discrepancy only appears on certain combinations of slicer values and is not reproducible consistently.

WRONG THINKING

Engineers review the DAX measures for errors and test them individually. Each measure looks correct when tested alone. The relationship configuration is confirmed to match documentation examples. The inconsistency feels like a DAX bug rather than a model design issue.

WHAT IS ACTUALLY HAPPENING

Many-to-many relationships with bidirectional cross-filtering produce filter propagation that is non-intuitive. When a slicer on the dimension table applies a filter, it propagates in both directions through the bridge table and can unintentionally filter both the fact table and other related tables. Measures that reference tables on both sides of the many-to-many relationship can receive double-filtered context that excludes rows that should be included in the total. This is not a bug. It is the expected behavior of bidirectional filtering applied to a many-to-many relationship, and it often produces results that contradict business logic.

THE FIX

Change the cross-filter direction of the many-to-many relationship from Both to Single, and use explicit DAX functions like `CROSSFILTER` inside measures that require bidirectional filtering only when needed. This prevents unintended filter propagation while still allowing measures to produce correct results when they explicitly request it. For complex many-to-many scenarios, consider restructuring the model to materialize the bridge table logic in the Lakehouse as a pre-computed fact table with explicit keys, eliminating the need for the bridge table in the semantic model entirely.

10

SEMANTIC MODEL DESIGN

You imported all columns from every table. Report performance is poor and the model size is huge.

THE SITUATION

A semantic model was built by importing all columns from all available Lakehouse tables without selecting specific columns. The model is technically correct and all reports work. But the model size is several gigabytes, refresh takes over an hour, and report load times are consistently slow even for simple card visuals on small date ranges.

WRONG THINKING

Engineers assume that importing more data gives Power BI more flexibility and that unused columns do not affect performance because they are simply not shown in visuals. The idea that unused columns in the model consume memory and slow down every query is not intuitive.

WHAT IS ACTUALLY HAPPENING

In Power BI and Direct Lake mode, every column in the model is loaded into or referenced by the VertiPaq engine regardless of whether it appears in any visual or measure. Wide tables with many string columns, high-cardinality ID columns, and columns with large value sets increase the model dictionary size, consume more memory during column segment loading, and slow down relationship joins. Even a column that no report ever uses contributes to the total memory footprint and the time required to frame or import the model. Every unnecessary column is a tax paid on every query.

THE FIX

Audit the semantic model and hide or remove all columns that are not referenced by any measure, used in any relationship, or needed for any report filter or slicer. Use the Model Explorer in Power BI Desktop to identify unreferenced columns. Remove high-cardinality identifier columns that are not used as join keys. For string columns with many distinct values, evaluate whether they need to be in the model at all or whether they can be replaced with a lower-cardinality lookup. A model with fewer, well-chosen columns will always outperform a model with every available column regardless of how good the DAX is.

11

SEMANTIC MODEL DESIGN

Your date table is not marked as a date table. Time intelligence functions are returning wrong results.

THE SITUATION

A semantic model includes a date dimension table with a date column. DAX measures using time intelligence functions like `TOTALYTD`, `SAMEPERIODLASTYEAR`, and `DATEADD` return incorrect results or blank values for certain date selections. The date table has the correct date range and the correct column data types. The measures look syntactically correct.

WRONG THINKING

Engineers debug the DAX expressions and test the measures with different date parameters. The date table and date column are confirmed to contain the expected values. The issue is intermittent and hard to reproduce consistently, so the date table configuration is not examined.

WHAT IS ACTUALLY HAPPENING

Power BI time intelligence functions require the model to have a properly marked date table with a continuous date column covering the entire date range used in the model. When a date table is not marked using the Mark as Date Table setting, Power BI uses internal auto date/time tables instead. If auto date/time is disabled but the table is not marked, time intelligence functions lose their reference point for period calculations. Additionally, gaps in the date table or date ranges that do not cover all fact table dates cause time intelligence functions to return blank or wrong values for dates outside the covered range.

THE FIX

Mark the date dimension table as a date table in Power BI by selecting the table and choosing Mark as Date Table from the Table tools menu, specifying the date column. Verify that the date table contains a continuous sequence of dates with no gaps and that the date range covers all dates present in the fact table including future dates if the model needs to handle them. Disable the auto date/time feature in model settings to prevent Power BI from creating hidden date tables that conflict with the explicit date dimension.

12

SEMANTIC MODEL DESIGN

Bidirectional relationships in your model are causing filters to spread in unexpected directions.

THE SITUATION

A semantic model was built with several bidirectional relationships to simplify DAX measures and allow slicers to filter across multiple tables without explicit `CROSSFILTER` calls. Reports work correctly for most use cases but certain measure combinations produce wrong totals. When a slicer on one table is active, measures on unrelated tables are also being filtered when they should not be.

WRONG THINKING

Engineers test each relationship individually and confirm each one is correct in isolation. The unexpected filtering only appears when multiple slicers are active simultaneously. The bidirectional setting is considered a convenience feature rather than a potential source of incorrect results.

WHAT IS ACTUALLY HAPPENING

Bidirectional cross-filtering means that a filter applied to one table automatically propagates to all related tables in both directions, following every relationship chain in the model. In a complex model with multiple bidirectional relationships, a single slicer can trigger filter propagation through 3 or 4 table hops, eventually filtering tables that have no logical business relationship to the slicer. This is amplified when measure context is added. The engine is technically doing what was configured. The configuration is causing logically incorrect filter context.

THE FIX

Set all relationships to single-direction cross-filtering by default and use bidirectional filtering only where it is explicitly required and tested. Use `CROSSFILTER` inside specific measures to enable bidirectional filtering only for the measure that needs it, rather than enabling it at the relationship level where it affects every measure in the model. Document every bidirectional relationship with the specific business reason it exists. If you cannot articulate why a relationship needs to filter in both directions, it should be single-direction.

13

SEMANTIC MODEL DESIGN

Your model has a snowflake schema. Slicer behavior is inconsistent at different levels of the hierarchy.

THE SITUATION

A semantic model uses a snowflake schema with three or more dimension levels (category → subcategory → product) before reaching the fact table. Slicers built on the top-level dimension produce different totals than slicers built on the bottom-level dimension, even when the selections should be logically equivalent. The matrix totals line up at the lowest grain but diverge at the higher grains.

WRONG THINKING

Because Scenario 12 is about bidirectional relationships, this is read as the same issue and engineers flip cross-filter directions hoping to fix it. The real cause is different: the relationships here are uniformly single-direction. The issue is that filter propagation now has to traverse multiple hops, and any missing key, type mismatch, or referential integrity gap at any one hop causes a subset of fact rows to drop out of the filter.

WHAT IS ACTUALLY HAPPENING

Filter propagation in a snowflake is a chain. A filter on the top-level table must propagate through every intermediate table before reaching the fact, and every hop performs a join on the relationship key. The deeper the snowflake, the more places this can break. Two distinct failure modes account for almost all of these problems. (1) **Referential integrity gaps.** An intermediate dimension has rows whose key does not exist on the next dimension down. Power BI's relationships use a left outer join from the many side to the one side; when the many side has an orphan key, the engine substitutes a blank virtual row, and filters from above never reach the affected fact rows. The fact total only looks correct when every key lines up through the entire chain. (2) **Data type mismatch on the join key.** When one side of a relationship is whole number and the other is text, Microsoft's own relationship troubleshooting documentation states that the relationship is allowed but filters will not find any matches to propagate. The relationship looks healthy in the model view; propagation silently produces zero matches at that hop, and everything downstream is missing. This is distinct from Scenario 12, which is about filter direction; this scenario is about chain length and key integrity.

THE FIX

Flatten the snowflake into a single wide dimension table at the Lakehouse layer. Pre-join category, subcategory, and product into one denormalized dimension before the semantic model sees them. This is one of the few cases where data duplication in the gold layer is worth it: VertiPaq compresses repeated dimension values heavily, so the storage cost is small and the filter behavior becomes one hop instead of three. If the snowflake must be preserved, audit each relationship's join key data types explicitly (Power BI does not warn you when text joins to whole number), and use DAX Studio to inspect the storage engine query that the slicer produces so you can see at which hop the row count is dropping. Do not enable Assume referential integrity to mask this. That property is only available on DirectQuery relationships in the same source group; where it is available, it switches the join to inner-join semantics, which hides the orphan rows you are actually trying to find.

14

SEMANTIC MODEL DESIGN

You built a composite model mixing Direct Lake and Import tables. The behavior depends on which Direct Lake flavor.

THE SITUATION

A team wants to combine a large Direct Lake fact table with several smaller Import lookup tables sourced from outside the Lakehouse. With Direct Lake on SQL, the model author cannot add Import tables alongside the Direct Lake tables at all. With Direct Lake on OneLake, the Import tables can be added but report performance is worse than either pure mode alone for any cross-storage measure.

WRONG THINKING

The two flavors of Direct Lake are treated as if they have the same composite behavior. Engineers assume that if Direct Lake on OneLake permits Import tables, the resulting model is just as fast as either pure mode. The hidden cost of every cross-storage query is not considered until reports go live.

WHAT IS ACTUALLY HAPPENING

The two flavors of Direct Lake behave differently here. Direct Lake on SQL does not support composite models in the model itself: you can only build a composite model on top of a Direct Lake on SQL semantic model in Power BI Desktop, which is a different construction. Direct Lake on OneLake is multi-source and does allow Import tables in the same model. When you query across an Import table and a Direct Lake on OneLake table, the engine must materialize the join at query time because the two tables are in different physical stores. This is not a fallback to DirectQuery; it is a cross-storage join that VertiPaq performs in memory, but it cannot use the same compressed segment-scan path that a pure Direct Lake or pure Import query uses. The penalty scales with the size of the Direct Lake side of the join.

THE FIX

First, identify which flavor you have: if the `DirectLakeBehavior` property in the model is greyed out, you are on Direct Lake on OneLake. The cleanest fix is to land the external lookup data in the Lakehouse as Delta tables (via a Dataflow Gen2 or a pipeline) so the whole model is pure Direct Lake on OneLake. If the lookup data genuinely cannot be moved (rare; almost everything can be landed once a day), keep the cross-storage joins small: Import tables in a composite Direct Lake on OneLake model should be narrow dimension tables, not anything that participates in row-level filtering or fact-table joins.

15

SEMANTIC MODEL DESIGN

Your semantic model refresh is taking hours. The underlying Lakehouse tables are not that large.

THE SITUATION

A Direct Lake semantic model refresh that previously completed in a few minutes now takes 2 to 3 hours. The Lakehouse Delta tables have grown but not dramatically. The refresh succeeds without errors but the duration is blocking downstream reports from showing current data during business hours.

WRONG THINKING

Engineers check the Lakehouse table sizes and confirm they are not exceptionally large. The refresh log shows it is spending most of its time on statistics computation rather than data loading. Adding more capacity is considered as the first fix before investigating what the refresh is actually doing.

WHAT IS ACTUALLY HAPPENING

Direct Lake semantic model refresh has two components: framing, which reads Delta metadata and takes seconds, and statistics computation, which analyzes column value distributions to build the statistics that DAX query planning depends on. When a table has a very large number of distinct values across many columns, or when the Delta table has accumulated many small Parquet files, statistics computation can take significantly longer than the actual data access time. The refresh is not slow because the data is large. It is slow because the engine is sampling and computing statistics across a fragmented file structure.

THE FIX

Run **OPTIMIZE** on all Lakehouse Delta tables referenced by the semantic model to compact small files into larger Parquet files before the refresh runs. With fewer, larger files the statistics computation samples the data more efficiently. If the model has many high-cardinality columns that are not used in any filter or slicer, remove them from the model to reduce the statistics surface area. Schedule the OPTIMIZE maintenance job to run immediately before the semantic model refresh in the pipeline so each refresh benefits from a compact table structure.

FABRIC ENGINEERING PATTERNS

Chapter 3

DAX

Performance

Eight scenarios covering why DAX measures that work correctly in Power BI Desktop behave differently in the Fabric service and how Fabric data sources change DAX execution behavior.

16

DAX PERFORMANCE

A DAX measure that works correctly is slow on large tables despite looking simple.

THE SITUATION

A measure written as a simple aggregation like `SUM` or `COUNTROWS` over a Direct Lake table works correctly and returns fast results on smaller date ranges. When a report uses the measure without any date filter on a full year of data, the measure takes 20 to 30 seconds to return. The measure expression itself has no visible complexity.

WRONG THINKING

Engineers assume the slowness is caused by the data volume and consider adding a default date filter to all pages as a workaround. The measure expression is not examined further because it appears simple. The execution engine behavior under Direct Lake is not considered.

WHAT IS ACTUALLY HAPPENING

Direct Lake loads column segments into the VertiPaq engine on demand during the first query that touches each segment. An unfiltered aggregation over a full year forces the engine to load all column segments for every partition of the Delta table before computing the result. The first query triggers this cold load which is expensive. Subsequent queries benefit from the warmed cache and return much faster. The measure is not slow because the expression is complex. It is slow because it is performing the initial column segment load for the entire table.

THE FIX

Add a default slicer or page-level filter on the date column to limit the initial data load to a recent period such as the last 30 or 90 days. This reduces the column segments loaded on first query. For reports that genuinely require full year aggregations, trigger a warm-up query by running the report once after each semantic model refresh before making it available to users. Consider pre-computing full year aggregations as calculated measures that use `CALCULATE` with explicit date ranges rather than relying on unfiltered scans of the full table.

17

DAX PERFORMANCE

Your CALCULATE with multiple filters is returning wrong results on certain slicer combinations.

THE SITUATION

A measure uses **CALCULATE** with two or more filter arguments to compute a specific business metric. The measure returns correct results when tested in isolation and with most slicer combinations. On certain combinations of two or more slicers applied simultaneously, the measure returns values that do not match the expected business logic. The discrepancy is hard to reproduce consistently.

WRONG THINKING

Engineers test the CALCULATE expression extensively in DAX Studio with various filter combinations. Most combinations return correct results. The failure on specific combinations looks like a data issue and the source data is checked first. The CALCULATE filter interaction behavior is not the first thing examined.

WHAT IS ACTUALLY HAPPENING

CALCULATE filter arguments interact in ways that are not always intuitive. When two filter arguments reference columns from the same table, CALCULATE applies them as a conjunction within the table context. When they reference columns from different tables, CALCULATE applies them independently and the filter context from each propagates separately through the model relationships. If a slicer from an external table is already filtering the model and a CALCULATE filter argument adds another condition on the same table, the interaction between the slicer context and the CALCULATE filter depends on whether **KEEPFILTERS** or **ALL** modifiers are used. Missing these modifiers produces results that appear correct for simple cases but fail for complex filter combinations.

THE FIX

Use DAX Studio to evaluate the measure under the exact filter context that produces the wrong result. Check the query plan to see which filters are being applied and in which order. Add explicit **ALL** or **KEEPFILTERS** modifiers to CALCULATE filter arguments where the interaction with external slicer context needs to be controlled.

Document the expected behavior of each CALCULATE filter argument with respect to slicer interactions and test every measure against at least three combinations of slicer values before releasing to production.

18

DAX PERFORMANCE

A time intelligence measure works correctly on desktop. It returns blank in the Fabric service.

THE SITUATION

A time intelligence measure using `TOTALYTD`, `SAMEPERIODELASTYEAR`, or `DATEADD` returns correct results in Power BI Desktop with a local data connection. When the same report is published to the Fabric service and the semantic model connects to the Lakehouse, the measure returns blank for all date selections. The date column data type is confirmed as Date in both environments.

WRONG THINKING

Engineers check the date table for gaps and confirm the dates are correct. They republish the report multiple times. Since the measure works on desktop, the service environment is blamed. The date table marking and auto date/time settings are not compared between the two environments.

WHAT IS ACTUALLY HAPPENING

The Fabric service semantic model has auto date/time disabled by default for Direct Lake models. In Power BI Desktop with a local connection, auto date/time may be enabled which creates hidden date hierarchy tables that time intelligence functions use. When published to the service with auto date/time disabled and the date table not explicitly marked, time intelligence functions lose their reference point. The date table in the model is present but is not recognized by the engine as the authoritative time intelligence source, so the functions return blank rather than an error.

THE FIX

Mark the date table explicitly in the semantic model using the Mark as Date Table option and specify the date column. Verify that auto date/time is disabled in both the Desktop file settings and the service model settings to avoid conflicting hidden date tables. After marking the date table, republish and test the time intelligence measures against a range of date selections. Also confirm that the date table covers all dates present in the fact table including dates in the future if the model uses forward-looking measures.

19

DAX PERFORMANCE

Your SUMX over a large table is timing out. SUM of a pre-aggregated column is instant.

THE SITUATION

A measure using **SUMX** iterates over a fact table with 200 million rows to compute a weighted calculation. The measure times out or takes several minutes to return in any visual. A colleague rewrites the same business logic as a **SUM** over a pre-computed column added to the Lakehouse table and it returns in under a second. The DAX logic is equivalent but the performance difference is enormous.

WRONG THINKING

Engineers assume the SUMX performance issue is a Power BI limitation and accept the pre-computed column as the only solution. The reason for the difference is not investigated because the workaround works and re-education is not prioritized.

WHAT IS ACTUALLY HAPPENING

SUMX is a row-by-row iterator. For every row in the table, it evaluates the expression inside the function and accumulates the result. On a 200 million row fact table, the DAX engine must evaluate the expression 200 million times even though the final result is a single number. In Direct Lake mode, this iterator pattern forces the engine to materialize row-level data into the DAX evaluation context rather than using the columnar aggregation path that **SUM** uses. The **SUM** over a pre-computed column uses native columnar aggregation which is orders of magnitude faster because it operates on compressed column segments directly without row-level iteration.

THE FIX

Move complex row-level calculations into the Lakehouse as computed columns in the Delta table rather than computing them in DAX at query time. Pre-compute the weighted value at load time so the semantic model only needs a **SUM** to aggregate it. For calculations that cannot be pre-computed because they depend on filter context, restructure the DAX to use columnar aggregation functions with **CALCULATE** rather than row iterators. Use DAX Studio to identify all measures using **SUMX**, **AVERAGEX**, or other X-suffix iterators on large tables and evaluate whether each can be rewritten as a pre-aggregated column.

20

DAX PERFORMANCE

A measure using FILTER returns correct results but causes every visual on the page to slow down.

THE SITUATION

A single measure on a report page uses **FILTER** to apply a complex conditional logic before aggregating. The measure returns correct results. But when this measure is on the page, all other visuals on the same page load significantly slower, including visuals that do not use this measure. Removing the measure from the page restores normal performance for all other visuals.

WRONG THINKING

Engineers assume that visuals are independent and that a slow measure should only affect the visual displaying it. The idea that one measure can impact all other visuals on the page is not intuitive and is not investigated. The measure is accepted as slow and the page performance degradation is attributed to a general page complexity issue.

WHAT IS ACTUALLY HAPPENING

FILTER in DAX creates a table iterator that evaluates a condition for every row in the filtered table. When **FILTER** is applied to a large table, it generates a temporary table in the DAX evaluation context. In Direct Lake mode, creating large temporary tables from iterator functions puts pressure on the VertiPaq memory cache. When the cache is under pressure, column segments that other visuals on the page need may be evicted to make room, forcing those visuals to reload their column segments on their next query. One memory-intensive measure causes cascading cache evictions that slow down all other visuals on the same page.

THE FIX

Replace **FILTER** with **CALCULATE** using native filter arguments wherever possible. **CALCULATE** with filter arguments uses the columnar filter path rather than creating a temporary row-level table, which is far more memory-efficient. For conditions that genuinely require **FILTER**, reduce the table being filtered by applying a narrower base filter first to reduce the row count before the complex condition is evaluated. Test the measure in DAX Studio with the Server Timings pane open to confirm the rewrite reduces the SE Query count and duration.

21

DAX PERFORMANCE

Your DAX measure returns different results depending on which visual it is placed in.

THE SITUATION

A measure produces one value when placed in a card visual, a different value when placed in a table visual with a row context, and yet another value in a matrix visual. All three visuals reference the same measure with no additional filters. The measure expression looks unambiguous and the discrepancy between visual types is unexpected.

WRONG THINKING

Engineers verify the measure expression and check for syntax errors. The expression looks correct in all three contexts. The data is verified to be consistent. The different values across visual types look like a Power BI rendering bug rather than a filter context issue.

WHAT IS ACTUALLY HAPPENING

The measure is producing different results because each visual type applies a different implicit filter context to every measure it evaluates. A card visual evaluates the measure in the context of all active page and report filters with no additional row context. A table visual evaluates the measure once for each row, injecting the row values as additional filter context. A matrix visual evaluates the measure at each cell intersection defined by row and column headers. A measure that does not explicitly control its own filter context using `ALL`, `ALLSELECTED`, or `CALCULATE` will produce different results in each of these contexts because the evaluation context is different.

THE FIX

Open DAX Studio, connect to the semantic model, and evaluate the measure using `EVALUATE ROW("Measure", [MeasureName])` to see the result in an empty filter context. Then evaluate it with explicit filters to understand how each visual context affects it. Use `HASONEVALUE` or `ISINSCOPE` inside the measure to detect which visual context it is being evaluated in and produce the appropriate result for each context. For measures that should always produce the same value regardless of visual context, add explicit `ALL()` wrappers to remove the injected row context.

22

DAX PERFORMANCE

A measure using ALLSELECTED behaves differently than expected when slicers interact.

THE SITUATION

A measure uses `ALLSELECTED` to compute a percentage of total relative to the current slicer selection. The measure works correctly when a single slicer is active. When two slicers from different tables are active simultaneously, the percentage denominator becomes inconsistent. Some rows show percentages that exceed 100 percent and others show values that do not add up to the expected total.

WRONG THINKING

Engineers test the measure with each slicer individually and confirm it works correctly in both cases. The failure only appears with both slicers active. The `ALLSELECTED` behavior with multiple slicers is not well understood and the inconsistency is attributed to a Power BI bug.

WHAT IS ACTUALLY HAPPENING

`ALLSELECTED` removes filters from the current filter context that were applied within the visual itself, while preserving filters that were applied from outside the visual such as slicers and page filters. When two slicers from different tables are active, `ALLSELECTED` preserves both slicer filters in the denominator context. If the relationship between the two slicer tables creates an asymmetric filter propagation pattern, the denominator is filtered differently for different rows in the visual, producing percentages that do not follow the expected pattern. This is correct DAX behavior but the results contradict the business expectation.

THE FIX

Use `CALCULATE([Measure], ALLSELECTED(table[column]))` explicitly specifying which slicer columns should be included in the denominator rather than relying on the implicit behavior of `ALLSELECTED`. This gives precise control over which external filters the denominator respects. Test the measure with every combination of slicers that will be available on the report before publishing. Document the intended behavior for each slicer combination and validate the measure output against manually computed expected values.

23

DAX PERFORMANCE

Your RANKX measure looked right in dev. In production it returns wrong ranks for some items.

THE SITUATION

A **RANKX** measure ranks products or customers by a metric within a category. On the dev dataset (a few hundred rows per category) the rankings are correct. On the production dataset (tens of thousands of rows per category) the rankings are wrong for some items. Some items that should rank first appear mid-table; some lower-ranked items appear at the top. The DAX expression is unchanged.

WRONG THINKING

The framing "wrong rankings at scale" pulls engineers toward thinking **RANKX** has a row-count limit or a performance threshold that breaks correctness. It does not. **RANKX** is deterministic at every row count. What scale does expose three pre-existing bugs in the expression that small data was hiding: ties, BLANK handling, and an incorrect ranking-table scope.

WHAT IS ACTUALLY HAPPENING

One of three things, often all three. (1) **Ties**. Production data has identical metric values across multiple items. The default behavior of **RANKX** is Skip: tied items share a rank and subsequent ranks are skipped accordingly. That mixes the visual order in ways that look wrong to a human eye. (2) **BLANKs**. Per Microsoft's DAX documentation, when the ranking expression evaluates to **BLANK** it is treated as 0 for numeric expressions. Items with no fact data therefore rank alongside any items whose metric is also 0, which on an ascending sort surfaces them at the top where empty rows look most wrong. (3) **Wrong ranking-table scope**. The expression uses **ALL(Product)** when the visual is showing one category at a time; the rank is computed against all products in all categories, not just the products in the current category. Small data made this invisible because there were few cross-category collisions.

THE FIX

Fix all three explicitly. For ties, two safe options. The modern one is the **RANK** window function: **RANK(DENSE, ORDERBY([metric], DESC, [id], ASC))** handles ties natively by adding a second sort column, and avoids the floating-point trap that **RANKX** has with decimal numbers. The classic **RANKX** pattern, per SQLBI, is to rank on a composite where the high column is shifted above the low column's maximum: **RANKX(table, [metric] * MAX(table[id]) + table[id])**, using the actual **MAX** of the tiebreaker column rather than a hardcoded multiplier. If **[metric]** is a decimal type, round or cast it first because **RANKX** can produce incorrect ranks on raw IEEE 754 decimals (use **ROUND([metric], 2)** or change the column to Fixed Decimal Number). For BLANKs: wrap the ranking expression with **IF(NOT ISBLANK([metric]), [metric])** so blank items are excluded rather than ranked alongside zeros. For scope: use **ALLSELECTED** when the visual is doing the filtering, **ALLEXCEPT(Product, Product[Category])** when ranking within a category, and pure **ALL** only when the rank is genuinely global. Test the fix on the production data, not on a sample, because samples almost never reproduce the tie distribution that exposes the bug.

04

FABRIC ENGINEERING PATTERNS

Chapter 4 Row Level Security

Six scenarios covering how Row Level Security behaves differently in Fabric depending on whether security is applied at the semantic model, SQL endpoint, or OneLake layer.

24

ROW LEVEL SECURITY

Your RLS role is defined correctly. Users are seeing data they should not have access to.

THE SITUATION

An RLS role was defined in the semantic model with a DAX filter expression restricting data by user email using `USERNAME()` or `USERPRINCIPALNAME()`. The role is assigned to the correct users in the Power BI service. Despite this, users report seeing data rows that belong to other users or regions that they should not have access to.

WRONG THINKING

Engineers verify the role definition and confirm the DAX expression is syntactically correct. They test the role using the View As Roles feature in Power BI Desktop and it appears to work correctly in testing. The failure in production with real users is attributed to a service configuration issue.

WHAT IS ACTUALLY HAPPENING

The RLS role filters the security table correctly but the relationship between the security table and the fact table has bidirectional cross-filtering enabled. The filter applied by RLS on the security table propagates in both directions, reaching the fact table through one path, but the bidirectional relationship also allows data from the fact table to filter back through the security dimension to other tables. In certain visual contexts, this reverse filter propagation can expose rows that the forward RLS filter was supposed to restrict. Additionally, if the relationship cardinality is set incorrectly, the filter may not propagate to all rows in the fact table as expected.

THE FIX

Set all relationships in the security filter propagation path to single-direction cross-filtering with the filter flowing from the security dimension toward the fact table. Verify the relationship cardinality is correct for your data model. After fixing relationships, test RLS using View As with several different user identities including edge cases such as users with no data, users with data in multiple regions, and users who belong to multiple roles. Never rely solely on the View As test with the role creator's own identity as it may not reflect the actual production filter behavior.

25

ROW LEVEL SECURITY

RLS is applied on a dimension table. Users can still see all fact data through a disconnected visual.

THE SITUATION

An RLS role applies a filter on a dimension table to restrict users to their assigned region. Most report visuals correctly show only the user's region data. But one visual on the report displays a measure that aggregates the entire fact table and shows the correct total for all regions, not just the user's region. The visual does not reference the dimension table that RLS filters.

WRONG THINKING

Engineers confirm the RLS role is active and working on other visuals. The disconnected visual is assumed to have its own data connection issue. The relationship between the dimension table and the fact table is not checked for the specific measure context used in the disconnected visual.

WHAT IS ACTUALLY HAPPENING

RLS filters propagate from the filtered table to related tables through the model relationships. If the visual uses a measure that references the fact table directly without any column from the dimension table in the visual context, the RLS filter on the dimension table must propagate to the fact table through the relationship for the restriction to apply. If the relationship cross-filter direction does not allow this propagation, or if the measure uses `ALL(FactTable)` or similar functions that remove the propagated filter, the RLS restriction is bypassed for that specific measure evaluation.

THE FIX

Audit every measure and visual on the report against the RLS filter propagation path. For measures that aggregate the fact table directly, verify that the RLS filter from the dimension table reaches the fact table through an active relationship with the correct filter direction. Use DAX Studio with the View As role feature to evaluate each measure independently under the RLS context and confirm the result matches expectations. Add `USERELATIONSHIP` or explicit filter arguments to measures that need to respect RLS but are not naturally receiving the filter through the model relationships.

26

ROW LEVEL SECURITY

Your RLS filter uses `USERPRINCIPALNAME()`. It works in Desktop but returns blank data in the service.

THE SITUATION

An RLS role uses `USERPRINCIPALNAME()` to filter data based on the signed-in user's email address. Testing with View As in Power BI Desktop returns correct filtered data. When published to the Fabric service, users assigned to the role see blank visuals or no data. The role assignment in the service is confirmed correct.

WRONG THINKING

Engineers re-test the role in Desktop after publishing and it still appears to work correctly. The blank data in the service for real users is attributed to a workspace permission issue rather than the RLS expression itself. The difference between how Desktop and the service resolve `USERPRINCIPALNAME()` is not considered.

WHAT IS ACTUALLY HAPPENING

In Power BI Desktop, `USERPRINCIPALNAME()` returns the UPN of the locally signed-in user, and View As substitutes a simulated identity. In the Fabric service, it returns the real authenticated UPN. Three things commonly cause a mismatch, in roughly this order of frequency. (1) **UPN does not match the security table.** The security table was built from HR email data, but the user's Entra UPN differs (rename, domain migration, never resynced). The values returned by `USERPRINCIPALNAME()` and the stored column look similar but are not equal. (2) **B2B guest UPNs.** External users invited as guests get a UPN of the form

`john.doe_othertenant.com#EXT#@yourtenant.onmicrosoft.com`, which never matches a plain email in the security table. (3) **Case sensitivity at the fallback path.** DAX string comparison is normally case-insensitive, but if the model is Direct Lake on SQL and the query has fallen back to DirectQuery, the comparison runs against the Warehouse, which may have case-sensitive collation. `john.doe@contoso.com` does not match `John.Doe@contoso.com` in that path.

THE FIX

Normalize defensively even though it should not be needed: wrap both sides of the comparison with `LOWER()` in the RLS expression and store the security table in lowercase. Build the security table from Entra UPN, not from HR email, because the two drift apart over time. For guest users, store the full `#EXT#` form, or filter on a separate stable claim (object ID) rather than UPN. To diagnose live, deploy a one-line diagnostic measure `UPN Diag = USERPRINCIPALNAME()` on a test page and have the affected user open it; the value they see is exactly what RLS is comparing against. Do not trust Desktop View As to validate any of this; it cannot reproduce the fallback path or the guest UPN format.

27

ROW LEVEL SECURITY

RLS is causing report queries to run 10x slower than without security applied.

THE SITUATION

A report performs well for users not subject to RLS. When RLS is applied and users access the same report, query times increase dramatically. A visual that loads in 1 second without RLS takes 10 to 15 seconds with RLS active. The RLS filter expression is simple and targets a small security table.

WRONG THINKING

Engineers assume RLS has inherent performance overhead and accept the degradation as unavoidable. The security table size is checked and confirmed to be small. The relationship between the security table and the fact table is not examined for performance impact.

WHAT IS ACTUALLY HAPPENING

RLS performance degradation is most commonly caused by the security filter propagating through a low-selectivity path. If the security table joins to the fact table through an intermediate table with a many-to-many or many-to-one relationship, the RLS filter must traverse this relationship for every query. When the filtered security table produces a large intermediate result before reaching the fact table, the VertiPaq engine cannot use its fast columnar aggregation path and must evaluate the filter row by row. Additionally, if the RLS expression uses complex DAX functions like `PATHCONTAINS` for hierarchy security, the evaluation cost multiplies with the number of rows in the security table.

THE FIX

Simplify the RLS filter expression to use direct equality comparisons on a single indexed column wherever possible. Avoid using DAX functions that require row-by-row evaluation inside RLS expressions. If the security table must join through multiple relationship hops to reach the fact table, create a pre-flattened security mapping table in the Lakehouse that directly links each user to their permitted fact table keys with a single relationship. Measure query times with and without RLS in DAX Studio to quantify the overhead and identify which part of the filter propagation path is most expensive.

28

ROW LEVEL SECURITY

You applied RLS on a Direct Lake on SQL model. It silently fell back to DirectQuery for all secured users.

THE SITUATION

A Direct Lake on SQL semantic model was performing well. RLS roles were added to restrict data access by user. After deploying RLS, users assigned to roles report that report performance dropped dramatically. Users without role assignments still experience fast Direct Lake performance. The model label still shows Direct Lake for all users.

WRONG THINKING

Engineers check the RLS role definitions and confirm they are syntactically correct. The performance difference between secured and unsecured users is attributed to the additional filtering overhead that RLS adds. The fundamental change in execution mode caused by RLS on a Direct Lake on SQL model is not known.

WHAT IS ACTUALLY HAPPENING

When a Direct Lake on SQL semantic model has RLS or OLS defined at the SQL analytics endpoint of the source (lakehouse or warehouse), every query from a role-bearing user must be validated against that source security. The engine cannot satisfy the query from compressed Delta segments alone because it does not know which rows the user is allowed to see until the SQL endpoint applies the security rules. The query path therefore switches to DirectQuery against the SQL endpoint for that user, while users without roles continue to read directly from Delta. The Direct Lake label in the model view does not change because the storage mode is unchanged at the model level; only the per-query execution path differs. This is the most common cause of "RLS made everything slow."

THE FIX

There are three honest options, in order of long-term value. (1) **Move RLS into the semantic model.** Define roles with DAX filter expressions in the model. The VertiPaq engine evaluates them in memory and Direct Lake stays on the in-memory path. This is the right answer for most teams but the migration is non-trivial: every existing role must be reauthored, regression-tested with real user accounts, and signed off by whoever owns access control. Budget weeks, not days. (2) **Move RLS into OneLake Security and switch to Direct Lake on OneLake.** This is Microsoft's current recommendation as of 2026 because OneLake Security applies across every Fabric compute engine, not just Power BI. It requires a model migration (see Scenario 07) and an organizational commitment to centralized access control. (3) **Accept the fallback.** If neither migration is feasible right now, keep SQL endpoint RLS but stop calling the model Direct Lake; it is effectively DirectQuery for secured users. Confirm `DirectLakeBehavior` is on its default value `Automatic` so the fallback actually runs; do not set it to `DirectLakeOnly` in this situation, because that would cause secured queries to fail outright instead of falling back. Plan one of the first two as real work, not a side-task.

29

ROW LEVEL SECURITY

Your RLS works for individual users. It breaks when users belong to multiple security roles.

THE SITUATION

RLS roles are defined for different regions. Users assigned to a single region role see correct filtered data. A small number of users who are assigned to two region roles simultaneously see either no data or data from only one of their assigned regions instead of the union of both. The role assignments in the service are confirmed correct.

WRONG THINKING

Engineers review each role individually and confirm each works correctly in isolation. The failure only appears for users with multiple role assignments. The union behavior of multiple Power BI roles is assumed to work correctly because it is documented as supported.

WHAT IS ACTUALLY HAPPENING

Power BI applies multiple roles using a union of their filter conditions when a user belongs to more than one role. This means a user assigned to both Region A and Region B roles should see data for both regions. However, if the roles are defined using conditions that reference different tables or use different filter approaches, the union may not produce the expected result. Additionally, if any one role includes an `ALL()` modifier or has no filter on a particular table, that role's empty filter effectively removes all restrictions for that table when unioned with a more restrictive role, potentially exposing more data than intended.

THE FIX

Review every RLS role definition for `ALL()` references or missing filters on tables that other roles restrict. When a user belongs to multiple roles, any role with no filter on a table effectively grants unrestricted access to that table for that user. Redesign roles so each role applies a complete and consistent set of filters across all restricted tables. Test multi-role behavior explicitly by assigning a test account to two roles simultaneously and verifying the resulting data access matches the expected union of permissions.

30

ROW LEVEL SECURITY

You are using Direct Lake on OneLake. Your RLS is defined in the semantic model. It is still causing performance issues.

THE SITUATION

A Direct Lake on OneLake model has RLS defined at the semantic model layer, which was the recommended approach after migrating from Direct Lake on SQL. Performance for secured users improved compared to the SQL endpoint RLS approach but is still noticeably slower than for unsecured users. The model is confirmed to be staying in Direct Lake mode for secured users.

WRONG THINKING

Engineers assume the remaining performance gap is unavoidable RLS overhead that cannot be eliminated. Since the model did not fall back to DirectQuery, the expectation is that performance should be equivalent to unsecured access. The specific cost of RLS filter evaluation within the VertiPaq engine is not investigated.

WHAT IS ACTUALLY HAPPENING

Semantic model RLS in Direct Lake on OneLake is evaluated inside the VertiPaq engine, which is the fast path. The remaining gap usually comes from one of three places. (1) **Security table cardinality.** A security table with one row per user-region-product combination can be tens or hundreds of thousands of rows; every query intersects against it. (2) **Relationship traversal in the role filter.** A DAX filter expression that walks two or three relationships to reach the fact table is expensive per query. (3) **Wrong layer entirely.** As of 2026, with OneLake Security now reaching general availability, Microsoft's documented best-practice guidance is to define access control in OneLake Security rather than semantic model RLS for data that is also queried by Spark, Warehouse, or KQL workloads, because the same roles apply across engines without per-engine duplication. There is one configuration step worth knowing: SQL analytics endpoints must be switched to User's identity access mode for OneLake Security to be evaluated per user (it is not the default; the default uses a delegated identity). Semantic-model RLS remains the right tool when report consumers do not have direct lakehouse permissions and the cloud connection uses a fixed identity.

THE FIX

Address the three causes in order. (1) Flatten the security table: one row per user with packed permission columns, joined directly to the fact table on a single user key. The compressed row count matters more than the schema purity. (2) Keep the role filter expression to a single hop. If the filter has to traverse a hierarchy, materialize the traversal as a column on the security table at the lakehouse layer, not at query time. (3) Decide where security actually belongs. If this data is also read by Spark notebooks, Warehouse queries, or anything outside Power BI, move the access control to OneLake Security and let it apply across engines; semantic model RLS becomes redundant. If this data is only ever read through Power BI by users with no lakehouse access, semantic model RLS is correct and you only have problems (1) and (2) to solve. Benchmark with DAX Studio server timings, secured versus unsecured, before and after each change.

FABRIC ENGINEERING PATTERNS

Chapter 5

Report and Visual Performance

Seven scenarios covering why reports that perform well in Power BI Desktop slow down dramatically in the Fabric service and how visual design choices affect query performance.

31

REPORT AND VISUAL PERFORMANCE

Your report page loads in 2 seconds on desktop. It takes 30 seconds in the Fabric service.

THE SITUATION

A report built and tested in Power BI Desktop loads quickly with acceptable performance. After publishing to the Fabric service and connecting the semantic model to the Lakehouse, the same report page takes 25 to 30 seconds to fully load. All visuals appear blank for an extended period before populating. The data and measures have not changed.

WRONG THINKING

Engineers assume the Fabric service has more latency than local desktop execution and accept the slower performance as an expected trade-off. The semantic model connection type is not examined and the difference between local Import mode in Desktop and Direct Lake mode in the service is not considered.

WHAT IS ACTUALLY HAPPENING

In Power BI Desktop with a local connection, data is typically in Import mode loaded into local memory, making queries nearly instantaneous. When published to the Fabric service with a Direct Lake connection to the Lakehouse, queries execute against Delta table column segments loaded from OneLake. The first page load triggers cold loading of all column segments needed by every visual on the page simultaneously. With 10 or more visuals all requesting data at the same time, the column segment loading requests compete and the page waits for all visuals to complete before rendering. Desktop performance gives no indication of this cold load cost.

THE FIX

Reduce the number of visuals on a single page to minimize simultaneous column segment load requests on page open. Implement report page loading states with a loading indicator so users understand the page is initializing. Use the Performance Analyzer in Power BI Desktop with a Direct Lake connection rather than a local Import connection to get accurate performance measurements. Consider pre-loading the most important visual first using bookmarks or a landing page design that shows critical KPIs before loading the full analytical page.

32

REPORT AND VISUAL PERFORMANCE

A matrix visual with many rows and columns is causing the entire report page to hang.

THE SITUATION

A matrix visual displaying a large cross-tabulation with dozens of row headers, multiple column groupings, and several measures causes the report page to become unresponsive. Other visuals on the page fail to load while the matrix is rendering. The matrix visual sometimes renders after several minutes or the page times out completely.

WRONG THINKING

Engineers optimize the DAX measures used in the matrix and reduce their complexity. Performance improves slightly but the matrix continues to cause page hangs for large row and column combinations. The matrix visual's inherent query behavior is not examined.

WHAT IS ACTUALLY HAPPENING

A matrix visual generates one DAX query per distinct combination of row context, column context, and measure. A matrix with 50 row values, 10 column values, and 5 measures generates up to 2500 individual cell queries in the worst case. In Direct Lake mode, each cell query requires the VertiPaq engine to evaluate the measure in a specific filter context. The concurrent evaluation of thousands of cell queries saturates the query execution threads and blocks other visuals on the page from receiving query slots. The page appears to hang because the matrix is consuming all available execution capacity.

THE FIX

Add row and column limits to the matrix visual using Top N filters to cap the displayed rows to a practical number such as 50 or 100. Enable virtual rendering in the matrix settings which loads only the visible cells rather than all cells upfront. For large cross-tabulations, consider replacing the matrix with a paginated report or an exported table that can handle arbitrary row counts without blocking interactive rendering. If the full cross-tabulation is a business requirement, pre-aggregate the matrix data in the Lakehouse as a summary table and build the matrix from the pre-aggregated table rather than from the raw fact table.

33

REPORT AND VISUAL PERFORMANCE

Your report has 15 visuals on one page. Every slicer interaction refreshes all visuals simultaneously.

THE SITUATION

A dashboard page has 15 visuals including charts, cards, tables, and KPIs. Every time a slicer is changed, all 15 visuals refresh simultaneously. The page takes 20 to 30 seconds to fully update after each slicer interaction. Users find the experience unusable for exploration.

WRONG THINKING

Engineers optimize the most complex visuals and reduce measure complexity. The overall refresh time improves slightly but the fundamental behavior of all visuals refreshing together on every slicer change is accepted as how Power BI works. Slicer interaction design is not considered as a performance lever.

WHAT IS ACTUALLY HAPPENING

By default, every visual on a Power BI page subscribed to the same slicer refreshes simultaneously when the slicer value changes. With 15 visuals, 15 DAX queries are submitted to the semantic model concurrently. In Direct Lake mode, 15 concurrent queries compete for column segment access and query execution slots. The total page refresh time is determined by the slowest visual, not the average, because Power BI waits for all visuals to complete before signaling that the page is updated. More visuals on a page always increases the worst-case refresh time.

THE FIX

Reduce the number of visuals per page to the minimum needed to answer the business question on that page. Use drill-through pages for detail views instead of putting all information on one page. Enable the Edit Interactions feature to configure which visuals respond to which slicers, disconnecting slicers from visuals where the relationship is not needed. Use the Performance Analyzer to identify which visuals take the longest to respond to slicer changes and prioritize optimizing or removing those specific visuals.

34

REPORT AND VISUAL PERFORMANCE

A card visual showing a single number is taking longer to load than a complex chart on the same page.

THE SITUATION

A report page has a bar chart showing sales by region and a card visual showing total sales. The bar chart loads quickly. The card visual showing a single aggregated number consistently takes 5 to 10 times longer to load than the bar chart. The card measure appears simpler than the chart measures.

WRONG THINKING

Engineers assume the card measure has a hidden complexity and spend time optimizing its DAX expression. After multiple rewrites the card remains slower than the chart. The visual type is not considered as a factor in query performance.

WHAT IS ACTUALLY HAPPENING

Card visuals in Power BI generate a query that asks the semantic model for a single value computed across the entire table without any grouping dimension. A bar chart query groups data by region and computes a measure per region, which allows the VertiPaq engine to use its native grouping and aggregation path efficiently. A card visual requesting a grand total without any dimensional grouping forces the engine to scan the entire column and aggregate without the benefit of pre-computed segment-level summaries. For Direct Lake tables, this unfiltered grand total scan can be more expensive than a filtered grouped aggregation because it bypasses some segment-level optimizations.

THE FIX

Rewrite the card measure to use an explicit **CALCULATE** with a date filter scoped to the relevant reporting period rather than an unfiltered all-time total. Pre-compute frequently needed grand totals as calculated measures that cache their results rather than recomputing from scratch on every page load. If the business requires an all-time grand total, consider materializing it as a single-row summary table in the Lakehouse and using a direct lookup rather than a full table scan in DAX.

35

REPORT AND VISUAL PERFORMANCE

Your report loads fast for some users and very slowly for others with identical filters and permissions.

THE SITUATION

Two users open the same report at the same time with the same slicer selections and the same RLS permissions. One user's report loads in 3 seconds. The other takes 45 seconds. Both users have the same Power BI license and workspace access. The report has no user-specific data differences. The performance gap is consistent across multiple sessions.

WRONG THINKING

Engineers check for differences in browser, network, and device between the two users. When these are confirmed to be similar, the performance gap is attributed to a Power BI service load balancing issue and a support ticket is raised. The semantic model capacity context is not examined.

WHAT IS ACTUALLY HAPPENING

Power BI in Fabric shares capacity across all semantic model queries in the workspace. When one user's query arrives, it may benefit from column segments already loaded in the VertiPaq cache from a previous query. The other user's query may arrive at a moment when the cache has been partially evicted due to other workspace activity, forcing their query to reload column segments from OneLake. The user who benefits from a warm cache sees fast performance. The user who triggers a cold load sees slow performance. This cache state variability produces consistent performance gaps between users who query at different moments relative to workspace activity.

THE FIX

Schedule a warm-up process that queries the most commonly used report pages and measures after each semantic model refresh, before the report is opened by business users. This ensures the column segments needed for the main reports are loaded into the VertiPaq cache before the first user query arrives. For consistently fast performance, increase the Fabric capacity tier to provide a larger in-memory cache that can hold more column segments simultaneously, reducing the probability of cache eviction between user queries.

36

REPORT AND VISUAL PERFORMANCE

Drill-through from a summary page to a detail page is extremely slow despite the detail table being small.

THE SITUATION

A summary page shows high-level KPIs and loads quickly. A drill-through action takes the user to a detail page that shows individual transaction rows for the selected item. The detail table has at most a few thousand rows after filtering. Despite the small result set, the drill-through page takes 15 to 20 seconds to load.

WRONG THINKING

Engineers check the detail page measures and queries. The measures look simple and the filtered table is confirmed to be small. The drill-through page design is compared to other pages and looks equivalent in complexity. The drill-through mechanism itself is not examined as a performance factor.

WHAT IS ACTUALLY HAPPENING

Drill-through in Power BI passes the filter context from the source visual to the destination page as a URL parameter. The destination page receives this context and applies it across all visuals simultaneously at page load. In Direct Lake mode, the destination page must cold-load all column segments for every table referenced by every visual on the detail page, constrained by the drill-through filter context. Even though the final result set is small, the column segment loading must complete for all columns of all referenced tables before any visual renders. Detail pages with many columns from wide fact tables are especially expensive to cold-load.

THE FIX

Limit the columns projected on the detail page to only those needed for the specific drill-through use case. Avoid using table visuals that show all columns from a wide fact table on drill-through pages. Create a dedicated narrow projection table in the Lakehouse that contains only the columns needed for the detail view, and build the drill-through page from this narrow table rather than from the full fact table. This reduces the column segment surface area that must be loaded on drill-through, dramatically reducing the cold load time.

37

REPORT AND VISUAL PERFORMANCE

Your paginated report runs fine for small date ranges. It times out on full year queries.

THE SITUATION

A paginated report connected to a Fabric Warehouse runs correctly and quickly for date ranges of one week or one month. When users request a full year of data or export the full dataset, the report times out. No error message indicates a specific limit and the timeout appears before the report renders.

WRONG THINKING

Engineers check the Warehouse query performance directly and confirm the underlying SQL query returns results in a reasonable time. The timeout is attributed to a paginated report service limit and a support ticket is raised requesting a limit increase.

WHAT IS ACTUALLY HAPPENING

Paginated reports in Fabric have a query execution timeout that applies to the entire report rendering cycle, not just the data retrieval query. For full year queries, the data retrieval may complete within the timeout but the report rendering, pagination calculation, and data formatting steps add additional time that pushes the total duration past the timeout threshold. Additionally, paginated reports fetch all data before rendering any pages, unlike interactive reports that render incrementally. A full year query returning hundreds of thousands of rows must complete the entire fetch before the first page of the report is visible.

THE FIX

Add a required date range parameter to the paginated report with a maximum span validated on the report parameter page. Prevent users from requesting ranges larger than a defined maximum such as one quarter. For full year exports, pre-aggregate the data in the Warehouse at the appropriate grain and build the paginated report from the aggregated table rather than from the row-level fact table. For genuinely large exports, use a Fabric pipeline to write the export to OneLake and provide the user with a download link rather than attempting to render the full dataset in the paginated report engine.

FABRIC ENGINEERING PATTERNS

Chapter 6

Deployment and Workspaces

Seven scenarios covering how deployment pipelines, workspace promotion, and semantic model updates break in Fabric in ways that only appear after promotion to production.

38

DEPLOYMENT AND WORKSPACES

You promoted a report from dev to prod using deployment pipelines. Data connections broke silently.

THE SITUATION

A report and semantic model were promoted from the development workspace to the production workspace using Fabric deployment pipelines. The promotion appeared successful with no errors. When production users open the report, visuals show errors or return no data. The semantic model in production is connected to the development Lakehouse rather than the Lakehouse.

WRONG THINKING

Engineers check the deployment pipeline configuration and confirm the report was promoted correctly. The connection issue is noticed when users report errors. Re-running the deployment is attempted but the connection continues to point to the development environment. The deployment pipeline parameter configuration is not immediately examined.

WHAT IS ACTUALLY HAPPENING

Fabric deployment pipelines promote the report and semantic model definitions but do not automatically remap data connections between environments. If deployment pipeline parameters are not configured to map development data sources to production equivalents, the promoted semantic model retains the connection string from the development workspace. The model appears to have deployed successfully because the deployment pipeline's job is to promote the artifact, not to validate that the connection target is correct for the destination workspace.

THE FIX

Configure deployment pipeline parameters before promoting. In the pipeline settings, map each development data source to its production equivalent so the connection is automatically updated during promotion. After every promotion, verify the semantic model connection in the production workspace settings before notifying users that the promotion is complete. Add a post-deployment validation step that runs a test query against the production semantic model and confirms it returns data from the Lakehouse, not the development one.

39

DEPLOYMENT AND WORKSPACES

Your semantic model in production is pointing at the dev Lakehouse after deployment.

THE SITUATION

A semantic model deployed to production connects to the wrong Lakehouse. After investigating the deployment pipeline, engineers discover that while the report was promoted correctly, the semantic model connection was not remapped. Every time the semantic model is refreshed in production, it reads from development data and overwrites the production semantic model cache with development values.

WRONG THINKING

Engineers manually update the connection in the production workspace to point to the Lakehouse. The fix works temporarily but the next deployment pipeline run reverts the connection back to the development Lakehouse again. The manual fix is applied repeatedly without understanding why it keeps reverting.

WHAT IS ACTUALLY HAPPENING

Each deployment pipeline run overwrites the semantic model definition in the target workspace with the definition from the source workspace. If the source workspace semantic model has the development connection hardcoded in its definition and deployment parameters are not configured to override it, every deployment resets the production connection to the development source. Manual connection updates in the target workspace are always overwritten by the next pipeline run because the pipeline considers the source definition authoritative.

THE FIX

Set up deployment pipeline parameters that explicitly map source workspace connections to target workspace connections. These parameters are configured at the pipeline level, not in the semantic model itself, so they survive every deployment run. Alternatively, use workspace-level connection overrides in the Fabric workspace settings to specify that any semantic model deployed to the production workspace should use the Lakehouse connection regardless of what connection is in the source definition. Test the parameter mapping by running a deployment and verifying the connection before any users access the model.

40

DEPLOYMENT AND WORKSPACES

A scheduled semantic model refresh is failing in production. Works when triggered manually.

THE SITUATION

A semantic model in the production workspace has a scheduled refresh configured. The scheduled refresh fails consistently with a credentials or connection error. When an administrator manually triggers a refresh on demand, it succeeds immediately. The error appears intermittently during scheduled runs but never during manual runs.

WRONG THINKING

Engineers check the workspace connection credentials and confirm they are valid. They re-enter the credentials and the next scheduled run succeeds. The failure returns on subsequent scheduled runs. The difference in identity between scheduled and manual refresh execution is not examined.

WHAT IS ACTUALLY HAPPENING

Manual refresh in the Fabric service runs under the identity of the signed-in user who clicks the refresh button. That user's credentials are used to authenticate to the data source. Scheduled refresh runs under the identity of the user who set up the schedule or a service principal if configured. If the scheduling identity's credentials or permissions have changed, expired, or are different from the administrator's credentials, the scheduled refresh fails while manual refresh by an admin continues to succeed. This difference is invisible in the UI because both show as refresh operations on the same semantic model.

THE FIX

Configure the scheduled refresh to use a service principal or a dedicated service account with a non-expiring credential rather than a personal user account. In the semantic model credentials settings, switch the authentication method from OAuth user credentials to service principal credentials. Verify the service principal has the necessary permissions on the Lakehouse or Warehouse being connected. After switching, run both a manual and a scheduled refresh to confirm both succeed under the service principal identity.

41

DEPLOYMENT AND WORKSPACES

You updated a semantic model. All reports built on it broke without warning.

THE SITUATION

A semantic model was updated to add new measures and rename several existing columns to align with a new naming convention. The update was published to the workspace. Within minutes, multiple reports built on the semantic model started showing errors. Visuals referencing renamed columns display field not found errors. No warnings were issued during the publish process.

WRONG THINKING

Engineers assume the column rename was a non-breaking change since the data is the same and only the display name changed. The impact on dependent reports is not anticipated because the semantic model publish process does not surface a dependency warning before the change takes effect.

WHAT IS ACTUALLY HAPPENING

Power BI report visuals bind to semantic model fields by their name, not by an internal identifier. When a column is renamed in the semantic model and the model is published, all reports that reference the old column name immediately lose their binding. The visual displays an error because the field name it was built with no longer exists in the model. There is no grace period, no warning dialog during publish, and no automatic rebinding. The rename is effectively a breaking change for all downstream reports the moment the updated model is published.

THE FIX

Before renaming any column or measure in a semantic model, audit all reports in the workspace that use that field using the semantic model's lineage view in the Fabric workspace. Either update all dependent reports simultaneously as part of a coordinated release, or use a phased approach: add the new column name alongside the old one, migrate reports to the new name, then remove the old name in a subsequent release. Never rename semantic model fields that have dependent reports without a coordinated update plan for those reports.

42

DEPLOYMENT AND WORKSPACES

Your deployment pipeline promoted changes correctly. End users are still seeing old data.

THE SITUATION

A deployment pipeline successfully promoted a semantic model update to production. The pipeline run shows green status. The semantic model in production shows the updated definition. But users report that reports are still showing data from before the promotion. The data in the Lakehouse is confirmed to be current.

WRONG THINKING

Engineers re-run the deployment pipeline hoping a second promotion clears the issue. When the second run does not change what users see, a semantic model refresh is triggered manually. After the refresh, users see current data. The connection between deployment and refresh is not immediately obvious.

WHAT IS ACTUALLY HAPPENING

Deployment pipelines in Fabric promote the semantic model definition but do not automatically trigger a semantic model refresh after promotion. A Direct Lake semantic model refresh reframes the model against the current Delta table state in the Lakehouse. Without a post-deployment refresh, the promoted model is still framed to the Delta table version that existed at the time of the last refresh before promotion. Users see the data as of that previous framing point until a new refresh explicitly updates the framing.

THE FIX

Add a semantic model refresh step at the end of every deployment pipeline run. This can be automated using a Fabric notebook activity or the Power BI REST API triggered after the deployment pipeline completes. Establish a deployment checklist that always includes a post-deployment refresh as a mandatory step before notifying users that the deployment is complete. For time-sensitive deployments, monitor the refresh completion before confirming to users that the updated report is live.

43

DEPLOYMENT AND WORKSPACES

A report that works for workspace members fails for external users with view access.

THE SITUATION

A report shared with external users through a workspace app or a direct share link works correctly for internal workspace members. External users who access the same report through the share link receive permission errors, see blank visuals, or are prompted to sign in repeatedly. The report contains no user-specific data filtering beyond standard RLS.

WRONG THINKING

Engineers confirm the share link is set up correctly and the external users have view permissions on the report. When the report works for internal users with identical permissions, the issue feels like a Fabric service authentication bug rather than a configuration problem.

WHAT IS ACTUALLY HAPPENING

External users accessing shared Power BI reports in Fabric may not have access to the underlying semantic model's data sources even though they have view permission on the report itself. View permission on a report gives access to the report artifact but the semantic model refresh and query execution require the underlying Lakehouse or Warehouse to be accessible to the identity resolving the queries. For guest users from external tenants, Fabric workspace data source permissions may not automatically extend to the Lakehouse, and Direct Lake framing may fail for external identities that do not have OneLake read access.

THE FIX

Grant external users explicit read access to the Lakehouse or Warehouse that the semantic model connects to, in addition to the report view permission. For Direct Lake models, external users need OneLake read permissions on the Delta tables referenced by the model. If external users should not have direct data source access, use an Import mode semantic model for externally shared reports so query execution does not require real-time data source access. Alternatively, embed the report using a service principal with appropriate permissions so external user identities do not need direct data source access.

44

DEPLOYMENT AND WORKSPACES

Your workspace is on Fabric capacity. Refresh limits are still being hit, and the CU bill is rising.

THE SITUATION

A workspace running on a Fabric F-SKU capacity (post-Premium migration) still hits semantic model refresh failures with errors indicating refresh limits or capacity throttling. The team expected the F-SKU to remove the refresh frequency restrictions that existed under Pro and Premium P-SKUs. Engineers confirm the workspace is assigned to a Fabric capacity at the expected SKU.

WRONG THINKING

The mental model is "we bought a bigger SKU, so the limits go away." Fabric F-SKUs do not work that way. They expose a Capacity Unit (CU) budget that every workload (refresh, query, pipeline) consumes from. The limits are not removed; they are pooled. Refresh failures are now a symptom of CU contention with everything else running on the same capacity, not of an arbitrary refresh-per-day cap.

WHAT IS ACTUALLY HAPPENING

Three constraints still apply on Fabric F-SKUs: concurrent refreshes per capacity, the per-query CU budget, and the smoothing window that lets short bursts spike above the SKU limit but throttles sustained overruns. When multiple semantic models refresh at the same time on the same capacity, they compete for CUs. If concurrent refreshes exceed the SKU's parallelism cap, additional refreshes queue and may eventually time out. Separately, Import-mode refreshes have very different CU profiles from Direct Lake framing operations: a Direct Lake reframe is metadata-only and cheap, an Import refresh that rehydrates the data is expensive. Workspaces that migrated tables from Import to Direct Lake without revising the refresh schedule are still paying the Import CU cost on the old schedules.

THE FIX

Open the Fabric Capacity Metrics App and look at the timeline of CU consumption against your SKU's smoothing limit. Find the refresh spikes; they are usually one or two large Import models running simultaneously. Stagger those refreshes across the day so they do not overlap, and convert what you can to Direct Lake so framing replaces full rehydration. For event-driven freshness, trigger the model refresh from the end of the data pipeline (see Scenario 02) rather than on a fixed schedule, so refreshes only happen when there is new data. If after all of that you are still throttled at peak, the SKU is genuinely too small for the workload and the right answer is to size up, not to engineer around it.

FABRIC ENGINEERING PATTERNS

Chapter 7

Power BI and Fabric Data Sources

Seven scenarios covering the specific behaviors that emerge when Power BI connects to Fabric Lakehouses, Warehouses, OneLake shortcuts, and the SQL analytics endpoint.

45

POWER BI AND FABRIC DATA SOURCES

Your Power BI report queries the Warehouse endpoint. Results differ from querying the Lakehouse SQL endpoint.

THE SITUATION

A semantic model connects to a Fabric Warehouse. A separate semantic model connects to the same data through the Lakehouse SQL analytics endpoint. Both models are built on the same underlying OneLake data. But reports on the two models return different aggregated values for the same measures and date ranges. Neither result is obviously wrong at first inspection.

WRONG THINKING

Engineers assume the data in the Warehouse and the Lakehouse must have diverged and investigate the pipeline that loads data into both. When the source data looks identical, the query engine differences between the two endpoints are not considered as a possible cause.

WHAT IS ACTUALLY HAPPENING

The Warehouse endpoint and the Lakehouse SQL endpoint are two different query engines applying two different optimization paths to the same underlying OneLake Delta data. The Warehouse engine applies its own statistics, query planning, and type coercion rules. The SQL analytics endpoint applies a different set of rules. For most queries the results are identical but edge cases involving implicit type conversions, NULL handling, date precision differences, and floating point rounding can produce different results from the same underlying data. These differences are not bugs in either engine but reflect the distinct behavior of two separate SQL execution paths.

THE FIX

Standardize on a single endpoint for each semantic model and document which endpoint is used and why. For reports that require identical results across teams, ensure all semantic models connect through the same endpoint. Add a cross-endpoint reconciliation step to the data pipeline that compares key aggregate values between the Warehouse and SQL endpoint periodically and alerts when they diverge beyond an acceptable threshold. Where precision is critical, use explicit CAST expressions in the SQL layer to enforce consistent type handling before the data reaches the semantic model.

46

POWER BI AND FABRIC DATA SOURCES

Data written to the Lakehouse is not appearing in Power BI reports despite a completed refresh.

THE SITUATION

A pipeline writes new data to a Lakehouse Delta table and the semantic model refresh completes successfully. The refresh timestamp in the Power BI service updates to reflect the current time. Reports queried immediately after the refresh still show yesterday's data. Querying the Lakehouse table directly in a notebook confirms the new data is present.

WRONG THINKING

Engineers trigger multiple additional refreshes hoping one will propagate the new data. When the data eventually appears after several refreshes, the issue is attributed to an eventual consistency delay in the Power BI service. The Direct Lake framing process is not examined as the source of the lag.

WHAT IS ACTUALLY HAPPENING

The semantic model refresh updated the framing metadata but the framing pointed to a Delta table snapshot that was captured before the pipeline finished writing all transaction log entries. In Delta Lake, a write operation commits a transaction log entry as its final step. If the framing occurs between the data file writes and the transaction log commit, the framing captures a Delta version that does not include the latest data. The refresh reports success because framing completed without error, but the model is framed to an older snapshot than intended.

THE FIX

Implement an explicit pipeline ordering that ensures the semantic model refresh is triggered only after the Delta transaction log commit is verified. Add a validation step in the pipeline that reads the latest Delta table version from the transaction log and confirms the expected row count before triggering the refresh. Use `DESCRIBE HISTORY tablename LIMIT 1` in a notebook to verify the latest transaction version and row count before initiating the Power BI refresh via the REST API.

47

POWER BI AND FABRIC DATA SOURCES

Your semantic model connects to a Fabric Warehouse. Refresh is slower than when it connected to the Lakehouse.

THE SITUATION

A semantic model was migrated from connecting to a Lakehouse via Direct Lake to connecting to a Fabric Warehouse. The business data is identical and the model schema is the same. But semantic model refresh time increased from a few seconds to several minutes after the migration. Report query performance also changed.

WRONG THINKING

Engineers assume the Warehouse has more overhead than the Lakehouse and consider migrating back. The specific refresh mechanism differences between Lakehouse and Warehouse connections in Direct Lake are not investigated before deciding.

WHAT IS ACTUALLY HAPPENING

Direct Lake connections to a Lakehouse use framing which reads Delta metadata directly from OneLake in seconds. Direct Lake connections to a Fabric Warehouse use a different path that goes through the Warehouse SQL layer to resolve table metadata and statistics before framing. This additional SQL layer adds latency to the framing process, making Warehouse-connected Direct Lake model refreshes inherently slower than equivalent Lakehouse-connected refreshes. The query performance difference reflects the same underlying distinction in how each source type serves column segment metadata to the VertiPaq engine.

THE FIX

If refresh speed is critical, consider whether the data needs to be in the Warehouse or whether a Lakehouse Delta table with the same schema would serve the reporting use case equally well. For data that genuinely belongs in the Warehouse due to SQL workloads or governance requirements, accept the longer refresh time as the trade-off for Warehouse-native data management. Schedule the Warehouse-connected semantic model refresh during off-peak hours to minimize user impact. If both a Warehouse and a Lakehouse copy of the data are acceptable, maintain the Lakehouse copy specifically for the Direct Lake semantic model refresh path.

48

POWER BI AND FABRIC DATA SOURCES

A Power BI report on a composite model is returning inconsistent totals across related visuals.

THE SITUATION

A composite model combines a Direct Lake fact table from the Lakehouse with an Import mode dimension table from an external source. Two visuals on the same report page show the same metric but from different perspectives. The totals in the two visuals do not agree with each other. Both visuals are built on the same measures and the data has not changed.

WRONG THINKING

Engineers check the measures for errors and verify the data in both source tables. The data looks consistent. The visual configurations are compared and look equivalent. The composite model storage mode mixing is not considered as a source of inconsistency.

WHAT IS ACTUALLY HAPPENING

In a composite model mixing Direct Lake and Import storage modes, queries that cross the storage mode boundary use DirectQuery to join the two sides. The DirectQuery path and the Direct Lake path can produce slightly different results for aggregate measures due to timing differences in how each engine evaluates filter context and handles NULL values and type boundaries at the join. When two visuals use the same measure but one has an additional dimension from the Import table in its context and the other does not, one visual uses the cross-storage DirectQuery path and the other uses the pure Direct Lake path, producing results from two different execution paths that may not agree at the decimal level.

THE FIX

Identify which visuals trigger cross-storage mode queries by using the Performance Analyzer and checking whether queries show DirectQuery execution for visuals that should be Direct Lake. Move dimension tables from Import mode into the Lakehouse as Delta tables and reference them through Direct Lake to eliminate the storage mode boundary. If external dimension data cannot be moved to the Lakehouse, accept that composite model totals may have minor precision differences and document this behavior for report consumers. Avoid placing visuals that use cross-mode measures alongside visuals that use pure Direct Lake measures on the same page where totals are expected to reconcile.

49

POWER BI AND FABRIC DATA SOURCES

Your OneLake shortcut data is visible in the Lakehouse but not in the Power BI semantic model.

THE SITUATION

A OneLake shortcut points to an external data source and appears correctly in the Lakehouse explorer. The shortcut table can be queried through a Fabric notebook. But when a Direct Lake semantic model is built on the Lakehouse, the shortcut table either does not appear as an available table or appears but returns no data when queried through the semantic model.

WRONG THINKING

Engineers verify the shortcut is active and the data is accessible through the notebook. The semantic model is rebuilt from scratch assuming a configuration error during the original model creation. The shortcut is re-added and the model is refreshed. The table continues to be inaccessible through the semantic model.

WHAT IS ACTUALLY HAPPENING

Direct Lake semantic models require tables to be Delta format tables registered in the Lakehouse metadata. OneLake shortcuts that point to non-Delta data sources such as ADLS, Amazon S3, or Google Cloud Storage expose the data in the Lakehouse file explorer but do not automatically register the data as Delta tables in the Lakehouse metadata catalog. Without a Delta table registration, the Direct Lake framing process cannot discover the table and it is not available for the semantic model. The notebook can query the shortcut directly using file path access which bypasses the table registration requirement.

THE FIX

Register the shortcut data as a Delta table in the Lakehouse metadata by either creating a Delta table on top of the shortcut data using a notebook with `spark.read.format('parquet').load(shortcut_path).write.format('delta').save(delta_path)`, or by using the Lakehouse table creation feature that converts shortcut data to a registered Delta table. After registration, refresh the semantic model to pick up the newly registered table. For shortcuts to Delta-format data from Azure Data Lake Storage, the registration may happen automatically depending on the file format and Lakehouse configuration.

50

POWER BI AND FABRIC DATA SOURCES

Your semantic model refresh succeeds but certain measures return blank after the refresh completes.

THE SITUATION

A semantic model refreshes without errors. The refresh timestamp updates correctly. Most measures return data as expected. But a specific subset of measures that reference particular tables return blank values after every refresh, even though the underlying Lakehouse tables contain data. Manually querying those tables in DAX Studio returns correct values.

WRONG THINKING

Engineers re-run the refresh multiple times. The blank measures persist after every refresh. The tables referenced by the blank measures are checked in the Lakehouse and confirmed to contain data. The semantic model is republished from Desktop hoping the republish resolves a corrupted model state. The blank measures return after the next refresh.

WHAT IS ACTUALLY HAPPENING

The measures returning blank are referencing tables that have not been processed in the semantic model even though the Lakehouse tables contain data. In Direct Lake mode, tables must be framed and their column statistics must be computed during the refresh process. If a table fails to frame due to an unsupported column type, a Delta table format issue, or a table that was recently added without a full process operation, the table is present in the model definition but has no column segments in the VertiPaq cache. Measures that reference an unprocessed table return blank because the engine has no column data to aggregate.

THE FIX

After a refresh that produces blank measures, connect to the semantic model using DAX Studio and run `EVALUATE INFO.VIEW.TABLES()` to identify which tables have zero rows or show an unprocessed state. For tables showing as unprocessed, investigate the Delta table in the Lakehouse for format issues such as corrupted transaction log entries, unsupported column types, or very large numbers of small files that exceed the SKU framing limit. Fix the underlying Delta table issue and trigger a full semantic model refresh. Add a post-refresh validation measure that counts rows in each critical table and alerts when any table returns zero rows unexpectedly.

CONCLUSION

Reports are the surface. The problems run deeper.

Across 50 scenarios, one pattern repeats. Power BI problems in Fabric rarely originate in the report layer. They originate in the semantic model design, the Direct Lake connection mode, the RLS configuration, or the deployment process. The engineers who diagnose problems fastest are the ones who check the execution mode first. Is Direct Lake actually running in Direct Lake or has it silently fallen back to DirectQuery? That single check eliminates half the performance complaints without touching a single DAX measure.

ALSO IN THE SERIES

Lakehouse and PySpark

- Your Delta table has millions of small files. OPTIMIZE is making it worse.
- Your notebook runs in 2 minutes alone. 20 minutes when triggered from a pipeline.

Architecture Patterns

- You put everything in one Lakehouse. Teams are stepping on each other.
- You have no deployment pipeline. A bad change went to production.

And more. Free at ssanjaychandra.com

A NOTE ON THIS RELEASE

This is the initial release of the Fabric Engineering Patterns series. The patterns and fixes reflect the platform as it stands today. Microsoft Fabric evolves quickly and some guidance may need updating as the platform matures. If you spot a technical inaccuracy, a pattern that has been superseded by a platform update, or a scenario that deserves a deeper treatment, your feedback is genuinely welcome. Reach out through any of the channels below.

ACKNOWLEDGEMENTS

Claude (Anthropic) was used as a writing assistant while putting this book together.

CONNECT

-  www.ssanjaychandra.com
-  linkedin.com/in/ssanjaychandra
-  ssanjaychandra@outlook.com

